Algoritmi di ordinamento

Raccolta Completa: 3 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

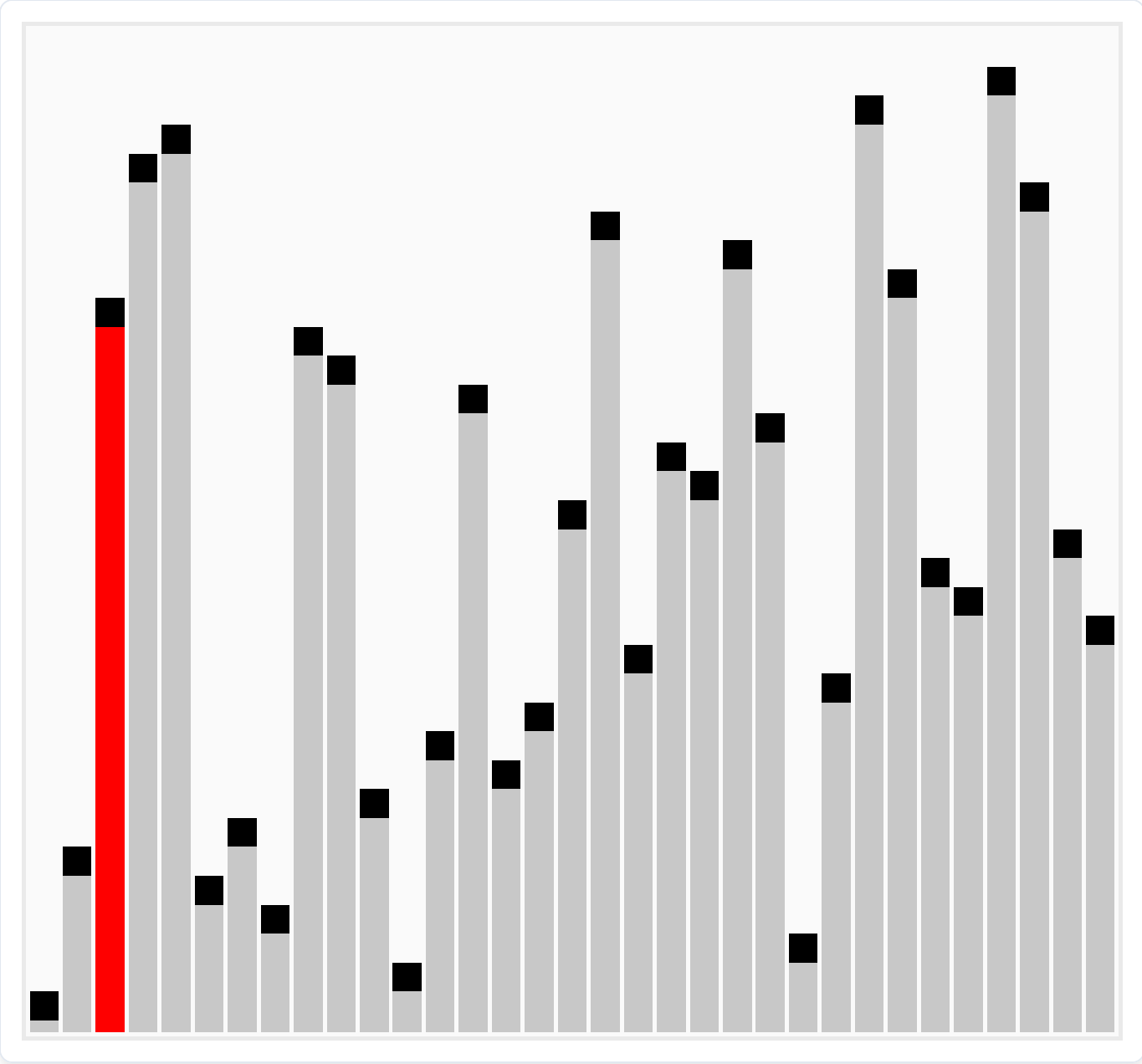
✨ Fonte Unica per Google NotebookLM

E’ un algoritmo semplice basato sul metodo degli scambi.

L’algoritmo confronta a due a due gli elementi adiacenti e li scambia tra loro. Se il primo elemento è maggiore del secondo quest’ultimo verrà spostato verso sinistra. Questo è il motivo anche di perché viene chiamato bubble sort, perché come le bolle la prima parte a essere ordinata e la parte maggiore dell’array.

Ripetendo N-1 volte questa operazione, tutti i valori risalgono verso destra fino ad occupare la posizione corretta e quindi ordinati in senso crescente.

Per ottimizzazione si mette una sentinella che controlla se ci sono state sostituzioni in un ciclo, se non ci sono state si blocca tutto.



Codice funzione bubble sort

```
void bubble_sort(int arr[], int size)
{
    int temp;
    int sentinella;
    for (int i = size - 1; i >= 0; i--) {
        sentinella = 0;
        for (int j = 0; j < i; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                sentinella++;
            }
        }
        if (sentinella == 0) {
            break;
        }
    }
}
```

Per testare:

```
#include <iostream>
#include <ctime>
#include <cstdlib>

using namespace std;

void bubble_sort(int arr[], int size)
{
    int temp;
    int sentinella;
    for (int i = size - 1; i >= 0; i--) {
        sentinella = 0;
        for (int j = 0; j < i; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                sentinella++;
            }
        }
        if (sentinella == 0) {
            break;
        }
    }
}

int main()
{
    srand(time(0));
    int numeri[50];
    int size = 50;
    cout << "Array non ordinato: ";
    for (int i = 0; i < size; i++) {
        numeri[i] = rand() % 1001;
        cout << numeri[i] << " ";
    }

    bubble_sort(numeri, size);

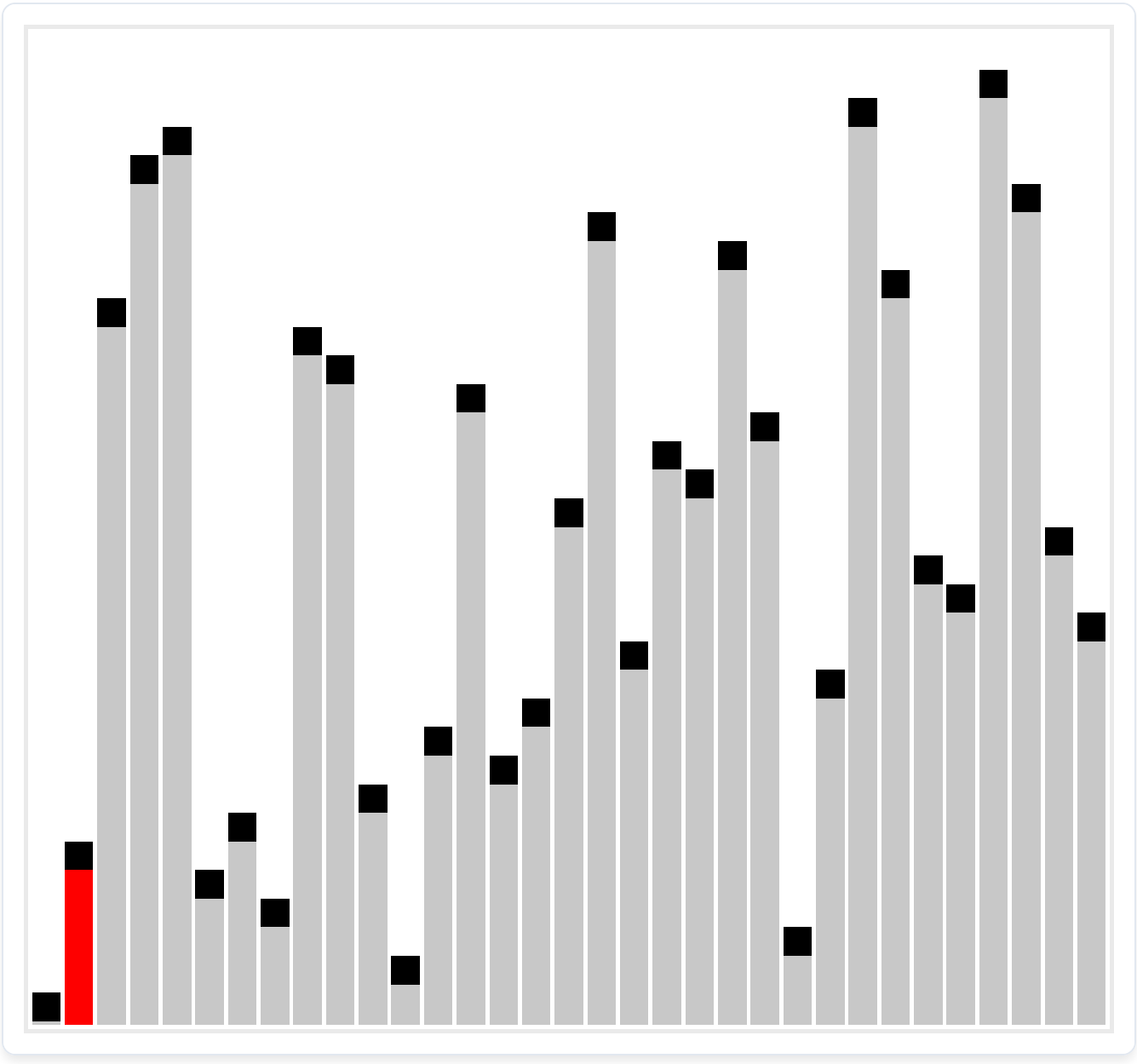
    cout << endl << endl << "Array ordinato: " << endl;
    for (int i = 0; i < size; i++) {
```

```
        cout << numeri[i] << " ";
    }
    cout << endl;
    return 0;
}
```

E’ più efficiente degli algoritmi visti in precedenza. Funziona efficientemente con array di dimensione molto piccola o con array parzialmente ordinati.

L’idea di ordinamento è simile al modo in cui un giocatore di bridge ordina le carte nella propria mano. Per trovare la giusta posizione si confronta la carta con le altre che sono nella mano da destra a sinistra.

Ogni carta più grande verrà spostata verso destra in modo da fare posto alla carta da inserire



Codice funzione Selection Sort

```
void insertion_sort(int arr[], int size)
{
    int temp;
    for (int i = 1; i < size; i++) {
        for (int j = i ; j > 0; j--) {
            if (arr[j]<arr[j-1]) {
                temp=arr[j];
                arr[j]=arr[j-1];
                arr[j-1]=temp;
            } else {
                break;
            }
        }
    }
}
```

Per testare:

```cpp
#include <iostream>
#include <ctime>
#include <cstdlib>

using namespace std;

void insertion_sort(int arr[], int size)
{
    int temp;
    for (int i = 1; i < size; i++) {
        for (int j = i ; j > 0; j--) {
            if (arr[j]<arr[j-1]) {
                temp=arr[j];
                arr[j]=arr[j-1];
                arr[j-1]=temp;
            } else {
                break;
            }
        }
    }
}

int main()
{
    srand(time(0));
    int numeri[50];
    int size = 50;
    cout << "Array non ordinato: ";
    for (int i = 0; i < size; i++) {
        numeri[i] = rand() % 1001;
        cout << numeri[i] << " ";
    }

    insertion_sort(numeri, size);

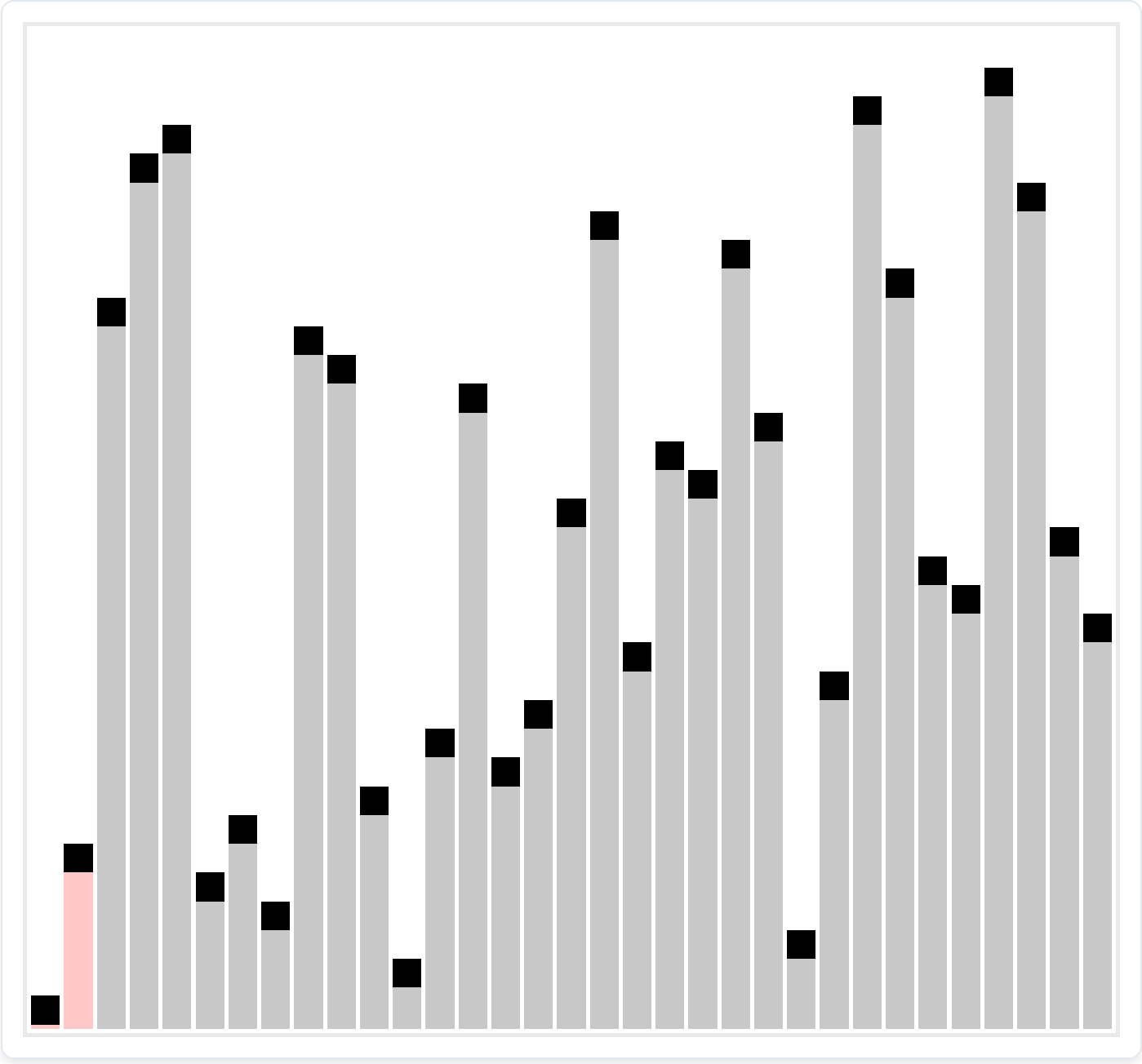
    cout << endl << endl << "Array ordinato: " << endl;
    for (int i = 0; i < size; i++) {
        cout << numeri[i] << " ";
    }
    cout << endl;
    return 0;
```

```
}
```

Il selection sort è un algoritmo di ordinamento di un array che va a dividere l'array in due, una parte iniziale ordinata e la seconda parte non ordinata da cui va a cercare sempre il minimo da inserire nella parte ordinata.

L'algoritmo sostanzialmente seleziona di volta in volta il numero minore nella parte non ordinata e lo sposta in quella ordinata, sostituendo il valore nella posizione i con il valore minore trovato nell'array non ordinato.

Con questo algoritmo la prima parte ad essere ordinata è l'inizio.



Codice funzione Selection Sort

```
void selection_sort(int arr[], int size)
{
    int temp;
    int min;
    for (int i = 0; i < size; i++) {
        min=i;
        for (int j = i; j < size; j++) {
            if (arr[j]<arr[min]) {
                min=j;
            }
        }
        temp=arr[min];
        arr[min]=arr[i];
        arr[i]=temp;
    }
}
```

Per testare:

```
#include <iostream>
#include <ctime>
#include <cstdlib>

using namespace std;

void selection_sort(int arr[], int size)
{
    int temp;
    int min;
    for (int i = 0; i < size; i++) {
        min=i;
        for (int j = i; j < size; j++) {
            if (arr[j]<arr[min]) {
                min=j;
            }
        }
        temp=arr[min];
        arr[min]=arr[i];
        arr[i]=temp;
    }
}

int main()
{
    srand(time(0));
    int numeri[50];
    int size = 50;
    cout << "Array non ordinato: ";
    for (int i = 0; i < size; i++) {
        numeri[i] = rand() % 1001;
        cout << numeri[i] << " ";
    }

    selection_sort(numeri, size);

    cout << endl << endl << "Array ordinato: " << endl;
    for (int i = 0; i < size; i++) {
        cout << numeri[i] << " ";
    }
    cout << endl;
```

```
    return 0;
}
```